
Desafio – Processamento de Remessas em Depósito

Atividade prática com consulta e prazo de entrega, baseada em problema do mundo real, onde os alunos devem utilizar:

- **Pilha** (*Stack*) implementada com uma **Lista Simplesmente Encadeada** (LSE);
- **Fila** (*Queue*) implementada com uma **Lista Duplamente Encadeada** (LDE);
- **Map**, mas especificamente **LinkedHashMap** do Java;
- **O algoritmo de Ordenação por Inserção**

Cenário

O depósito logístico da CEASA Aracaju recebe diariamente remessas de mercadorias oriundas de outros municípios, cada remessa pode conter vários itens como cebola roxa, cebola branca, batata, uva, laranja etc. Cada item possui três atributos principais:

- um identificador único literal ex.: (*CebolaRoxa, Uva, Batata etc.*);
- uma quantidade numérica; e,
- uma data de validade.

As remessas chegam à área de recebimento na ordem em que são entregues. O depósito armazena os itens em áreas dedicadas para cada identificador único, onde os itens devem ser organizados de forma que os que têm data de validade mais próxima sejam sempre os mais acessíveis. Periodicamente, a equipe do depósito deve processar todas as remessas recebidas, integrando os novos itens ao estoque existente e garantindo que essa organização por data de validade seja mantida para cada identificador único. Para realizar o processamento das Remessas o CEASA faz:

- As novas remessas devem ser processadas exatamente na ordem em que chegam;
- Cada tipo de item (identificador único literal - ID) deve ter sua própria área de armazenamento dedicada;
- Dentro do armazenamento de cada tipo de item (identificador único literal - ID), os itens devem ser organizados de forma que as datas de validade mais próximas sejam sempre as mais acessíveis para recuperação;
- Ao adicionar novos itens ao armazenamento existente, o sistema deve manter a organização baseada na data de validade.

Restrições:

1. Não devem ser usados vetores ou armazenamento externo: O processamento de remessas deve ser feito usando fila e pilha em conjunto com **LinkedHashMap** quando necessário.
2. A solução deve lidar com remessas com quantidades de itens variáveis.



Casos de uso:

- **Receber Remessas**

Método: *void receberRemessa(IItemEstoque[] remessa);*

Quando novas remessas chegam ao depósito, o método recebe a remessa e a coloca em uma área de armazenamento designada para processamento futuro. A remessa contém vários itens, cada um com seu próprio código de produto, quantidade e data de validade.

- **Processar Todas as Remessas**

Método: *void processarTodasRemessas();*

Realiza uma organização completa do armazenamento, recebendo todas as remessas recém-chegadas e integrando-as sistematicamente ao sistema de armazenamento do depósito. Ele processa cada remessa na ordem exata em que foi recebida, garantindo um tratamento justo por ordem de chegada.

- **Processar Uma Única Remessa**

Método: *void processarUmaUnicaRemessa(IItemEstoque[] remessa);*

Processa uma única remessa completa por vez, pegando todos os itens de uma única remessa e colocando-os cuidadosamente em seus respectivos locais de armazenamento. Isso ocorre após o recebimento da remessa, mas antes de sua integração completa ao sistema de inventário. O método deve implementar o algoritmo de ordenação por inserção. Veja qual parte específica realiza a ordenação. Esse método deve ser usado dentro do método *void processarTodasRemessas();*

- **Obter Status do Armazenamento**

Método: *String obterStatusArmazenamento();*

Gera um *snapshot* completo de todas as mercadorias atualmente armazenadas no depósito, mostrando exatamente como os itens estão organizados para cada tipo de produto. Isso proporciona aos gerentes e funcionários do depósito uma visão clara e imediata da organização do estoque e da sequência de item com data de validade mais próxima.

- **Recuperar Próximo Item da Remessa**

Método: *IItemEstoque recuperarProximoItemDaRemessa(String ID);*

Gerencia o processo de retirada de um lote específico de um tipo de item (ID) do armazenamento para distribuição ou uso. Ele garante que o item retirado seja sempre aquele que precisa ser usado com mais urgência, com base em sua data de



validade, ou seja, ele não retira qualquer item disponível - retira especificamente o lote mais próxima da data de validade do tipo de item solicitado.

- **Verificar Se Tem Remessas Recebidas Pendentes de Processamento**

Método: *boolean temRemessasRecebidasPendentesDeProcessamento();*

Fornece uma resposta rápida de "sim ou não" para uma pergunta simples: "Há alguma remessa recém-chegada aguardando processamento e armazenamento?" Não analisa, organiza ou altera nada – apenas fornece uma resposta clara e imediata sobre a existência de remessas não processadas, ajudando a manter as operações do depósito.

Considerações Adicionais

- Remessas Parciais: Uma única remessa pode conter vários tipos de item (ID) em quantidades variáveis;
- Espaço Vazio: Alguns tipos de item (ID) podem ser novos e não possuir armazenamento/estoque prévio;
- Remessas Futuras: O sistema deve estar preparado para lidar com novas remessas que cheguem mesmo durante o processamento;
- Simulação de Retirada: Durante as operações normais, os itens são sempre retirados da posição mais acessível (próximo item - data de validade mais próxima).

Entregáveis (o que o aluno deve entregar)

1. Código-fonte implementando:
 - a. A classe ***GerenciadorRemessaDeposito.java*** com a lógica principal do programa (casos de uso) dentro do ***package Laboratorio.desafios.remessasemdeposito*** no projeto ***algoritmos-estrutura-dados-java-lab***
2. Breve explicação curta (até 3 minutos) em formato de vídeo (exibindo o código) explicando:
 - a. Onde a estrutura de pilha foi usada e o motivo;
 - b. Onde a estrutura de fila foi usada e o motivo;
 - c. Como ***Map***, mas especificamente ***LinkedHashMap*** do Java foi usado e em conjunto com qual estrutura de dados e o motivo;
 - d. Qual a razão do uso algoritmo de ordenação por inserção.



Resultado Esperado

Ao executar o *método main* da *classe AppGerenciadorRemessaDeposito* o resultado deve ser parecido com a imagem abaixo.

```
#####  
### Gerenciador de Remessas em Deposito - CEASA ARACAJU ###  
#####  
  
[Etapa 01 - Recebendo as Remessas de Mercadorias Oriundas de Lagarto, Itabaiana e Capela Em Ordem de Chegada]  
  
1º Remessa de Lagarto Recebida com Dois Itens (Cebola Roxa e Batata)  
2º Remessa de Itabaiana Recebida com Dois Itens (Cebola Roxa, Cebola Branca e Batata)  
3º Remessa de Capela Recebida com Três Itens (Cebola Roxa, Laranja e Uva)  
  
[Etapa 02 - Status Do Armazenamento no Depósito Antes do Processamento das Remessas]  
  
Tem Remessas Recebidas Pendentes de Processamento? SIM  
Itens Armazenados no Depósito? NÃO  
  
[Etapa 03 - Processando Todas as Remessas]  
  
Todas as Remessas Processadas  
  
[Etapa 04 - Status Do Armazenamento no Depósito Após o Processamento das Remessas]  
  
Tem Remessas Recebidas Pendentes de Processamento? Não  
Itens Armazenados no Depósito? SIM  
  
- CebolaRoxa: [Quantidade: 500 , Data Validade: 02/03/2026] -> [Quantidade: 350 , Data Validade: 23/04/2026] -> [Quantidade: 100 , Data Validade: 24/06/2026]  
- Batata: [Quantidade: 100 , Data Validade: 14/05/2026] -> [Quantidade: 150 , Data Validade: 14/05/2026]  
- CebolaBranca: [Quantidade: 100 , Data Validade: 06/02/2026]  
- Laranja: [Quantidade: 120 , Data Validade: 25/12/2026]  
- Uva: [Quantidade: 1000 , Data Validade: 20/05/2026]  
  
[Etapa 05 - Recuperando Próximo Item (Data de Validade Mais Próxima) de Uma Determinada Remessa Armazenada no Depósito]  
  
Recuperando Próximo Item da Remessa de Cebola Roxa  
- Recuperando do Armazenamento: [Quantidade: 500 , Data Validade: 02/03/2026] (Data de Validade Mais Próxima: 02/03/2026)  
  
Recuperando Próximo Item da Remessa de Cebola Roxa  
- Recuperando do Armazenamento: [Quantidade: 350 , Data Validade: 23/04/2026] (Data de Validade Mais Próxima: 23/04/2026)  
  
Recuperando Próximo Item da Remessa de Uva  
- Recuperando do Armazenamento: [Quantidade: 1000 , Data Validade: 20/05/2026] (Data de Validade Mais Próxima: 20/05/2026)  
  
Tentando Recuperar de Uma Remessa Que Não Existe - Melancia  
- Recuperando do Armazenamento: Não Existe  
  
[Etapa 06 - Verificando Status do Armazenamento Após Recuperação de Itens Com Data de Validade Mais Próxima da Remessa de Cebola Roxa e Uva]  
  
Itens Armazenados no Depósito? SIM  
  
- CebolaRoxa: [Quantidade: 100 , Data Validade: 24/06/2026]  
- Batata: [Quantidade: 100 , Data Validade: 14/05/2026] -> [Quantidade: 150 , Data Validade: 14/05/2026]  
- CebolaBranca: [Quantidade: 100 , Data Validade: 06/02/2026]  
- Laranja: [Quantidade: 120 , Data Validade: 25/12/2026]  
- Uva:
```

Obs.: A classe *AppGerenciadorRemessaDeposito* não deve ser alterada, ela vai servir como teste manual. Assim como as interfaces *IItemEstoque* e *IGerenciadorRemessaDeposito* e a classe *ItemEstoque* também não devem ser alteradas. Você apenas deve alterar a classe *GerenciadorRemessaDeposito* para atender aos casos de uso e em conformidade com a interface.



Critérios de Avaliação

- **Corretude:** casos de uso devem funcionar conforme especificado.
- **Compleitude:** Todos os casos de uso necessários são implementados.
- **Qualidade do código:** Bem estruturado, comentado e organizado logicamente e modularizado quando necessário.

Objetivo Pedagógico da Atividade

- Conhecimento
 - Conceitual: os alunos devem entender por que uma pilha se adapta aos processos *LIFO* e uma fila se adapta aos processos *FIFO*.
 - Conceitual: como combinar uma estrutura de dados implementada no projeto com *Map/LinkedHashMap* para atender uma necessidade de negócio.
 - Procedural: os alunos devem praticar como usar essas estruturas de dados e o como implementar um algoritmo de ordenação.
- Taxonomia de *Bloom*
 - Aplicar: os alunos devem aplicar conceitos de estrutura de dados a um cenário real.
 - Analisar: Os alunos devem analisar como atender aos casos de uso a partir das estruturas de dados de fila e pilha já existentes e combinando com outras estrutura pré-existente em Java (Ex.: *Map/LinkedHashMap*).
 - Criar: Os alunos devem implementar os casos de uso em um projeto já existente e produzindo um código de solução original.
- Seleção e Justificativa da Estrutura de Dados
 - Objetivo: Os alunos demonstram capacidade de analisar os requisitos do problema e selecionar métodos apropriados de organização de dados com base em características comportamentais (ordenação, padrões de acesso, restrições de processamento) sem instruções explícitas.
- Implementação de Contrato de Interface
 - Objetivo: Os alunos comprovam a compreensão da abstração construindo soluções com base nas interfaces fornecidas (*IPilha*, *IFila*), respeitando os contratos operacionais (semântica *LIFO/FIFO*).



- Integração de Algoritmos
 - Objetivo: Os alunos demonstram a capacidade de incorporar lógica de ordenação em operações estruturais usando as estruturas já existentes.
- Orquestração de Sistemas Complexos
 - Objetivo: Os alunos projetam fluxos de trabalho coordenados onde múltiplas estruturas e métodos interagem para resolver problemas holísticos, como por exemplo (*Recebimento de Remessas → Verificação de Status → Processamento de Remessas → Verificação de Status → Recuperação de Próximo Item → Verificação de Status*).
- Desenvolvimento Orientado por Comportamento
 - Objetivo: Os alunos validam soluções em relação a requisitos funcionais em vez de detalhes de implementação.

Comentários Finais

Esta atividade fornece uma abordagem prática, integrando três conceitos fundamentais — pilhas, filas e ordenação básica — em um cenário prático de processamento de remessas de um depósito. Não é trivial nem excessivamente complexo, o que o torna adequado para uma tarefa de dificuldade média em um curso de estruturas de dados.

Ao fazer isso, garanta que os alunos pratiquem:

- Entendimento conceitual do motivo pelo qual certas estruturas de dados são escolhidas para tarefas específicas,
- Detalhes de implementação processual (*empilhar/desempilhar, enfileirar/desenfileirar*, etapas de classificação/ordenação) e
- Resolução de problemas de um domínio específico.